

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Aryan Arun	Reg. No.:	192411157
Section / Batch:	BE CSE / 2024	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Code of Conduct

I _____Aryan Arun (192411157)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____Aryan Arun_____

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				

Total Marks (100):	
---------------------------	--

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CSA0624 - Design and Analysis of Algorithms

Case Study Assessment — Divide and Conquer

Merge Sort · Quick Sort · Binary Search · Matrix Multiplication

Name: Aryan Arun

Reg.No: 192411157

Case Study 1: E-Commerce Product Sorting System (Merge Sort)

Scenario: An e-commerce platform must sort millions of products by price and ratings to ensure fast user access.

Analysis: How Merge Sort Handles Large Datasets Efficiently

Merge Sort is well matched to this scenario because its performance is completely independent of how the product data is initially arranged — whether products arrive newly listed, price-updated, or already partially sorted from a previous run, Merge Sort always splits and merges in exactly the same guaranteed pattern. For a catalog of millions of products, this predictability matters more than raw best-case speed: the platform cannot risk an unlucky data arrangement silently degrading sort time during a high-traffic sale event.

Key Issues Identified

Stability: Products are commonly sorted by more than one criterion — for example, primarily by price and, among equally priced items, by rating, or by a 'featured' flag. If the sorting algorithm is not stable, a secondary ordering (such as previously computed relevance rank) can be silently destroyed whenever two products tie on the primary key. Merge Sort is stable by construction (the merge step always takes from the left run first on a tie), which directly protects this multi-criteria ordering requirement.

Scalability: Millions of products may exceed available RAM, especially when re-sorting the full catalog rather than a single page of results. A purely in-memory algorithm risks running out of memory or thrashing. Merge Sort's structure — repeatedly merging sorted runs — extends naturally to external (disk-based) sorting: sorted chunks that each fit in memory can be produced independently and then merged in a streaming fashion, without ever needing the entire dataset in memory at once.

Applying Divide and Conquer

- **Divide:** the product list is recursively split into two halves by position, continuing until each sub-list contains a single product (trivially sorted).
- **Conquer:** each half is sorted independently and recursively using the same splitting process.
- **Combine:** two adjacent sorted halves are merged by repeatedly comparing their front elements (by price, then by rating as a tiebreaker) and appending the smaller to the result, until both halves are exhausted.
- This divide-merge pattern repeats until the entire product catalog is merged back into one fully sorted list.

Time Complexity and Performance Analysis

Case	Complexity	Reasoning
Best Case	$O(n \log n)$	The split is always by position, not by value, so even an already-sorted catalog still goes through the full $\lceil \log_2 n \rceil$ levels of division and merging.
Average Case	$O(n \log n)$	Identical structure regardless of the price/rating distribution across products.
Worst Case	$O(n \log n)$	No input arrangement can force extra recursion levels or extra merge work — a critical guarantee for a production system.

Space complexity is $O(n)$ due to the temporary arrays needed during merging — an acceptable trade-off given the platform's servers can allocate proportional memory, and this cost is what enables the external-sorting scalability discussed above.

Design and Justification

Proposed design: maintain the product catalog in manageable sorted chunks (e.g., per category or per server shard) using in-memory Merge Sort for each chunk, then merge chunks together using a k-way merge (an extension of Merge Sort's two-way merge) whenever a global, cross-category sorted view is needed — such as a 'best products across all categories' page. For real-time updates (new listings, price changes), newly added products can be merged into the

existing sorted structure incrementally rather than re-sorting from scratch, since Merge Sort's merge step is naturally suited to combining a small sorted batch with a large already-sorted list.

Why Merge Sort is suitable: it offers a guaranteed $O(n \log n)$ bound with no risk of worst-case collapse (unlike Quick Sort under certain data patterns), it is stable — essential for correctly preserving secondary sort criteria like rating or relevance — and its divide-and-merge structure scales naturally from in-memory sorting to distributed, disk-based, or parallel sorting across multiple servers, which is exactly the kind of infrastructure a large e-commerce platform operates on.

Case Study 2: Real-Time Ranking System (Quick Sort)

Scenario: A real-time ranking system updates player/user scores dynamically and needs fast sorting for leaderboard display.

Analysis: Applying Quick Sort to Real-Time Ranking

A leaderboard is re-sorted frequently — potentially every time a score changes — and users expect near-instant updates. Quick Sort's low constant-factor overhead and in-place partitioning make it attractive for this kind of frequent, in-memory, latency-sensitive resorting, where average-case speed matters more than worst-case guarantees, provided the worst case can be reliably avoided.

Key Issues Identified

Pivot selection: The naive strategy of always choosing the first or last element as the pivot performs poorly precisely on the kind of data a leaderboard tends to produce: scores that are already sorted, or nearly sorted, from the previous update cycle. On such input, a fixed-position pivot repeatedly creates a maximally unbalanced partition, degrading performance sharply.

Worst-case performance: With a poor pivot strategy, Quick Sort degrades from $O(n \log n)$ to $O(n^2)$ — for a live leaderboard serving continuous updates, this is not just a slow single event but a recurring risk every time the underlying score list happens to be close to sorted, which is common right after a previous sort.

Applying Divide and Conquer

- **Divide:** select a pivot score and partition the list so that all scores less than the pivot come before it and all scores greater come after (or vice-versa, depending on ranking direction).
- **Conquer:** recursively apply the same partitioning process to the sub-list before the pivot and the sub-list after the pivot.
- **Combine:** no explicit combine step is needed — once every element has been placed relative to its pivot at every recursive level, the list is sorted in place.

Efficiency Under Different Cases

Case	Complexity	Trigger Condition
Best Case	$O(n \log n)$	Pivot consistently splits the list into two roughly equal halves.
Average Case	$O(n \log n)$	Typical for a well-chosen pivot strategy across varied real-world score distributions.
Worst Case	$O(n^2)$	Pivot consistently picks the smallest or largest remaining element — triggered reliably by sorted or nearly-sorted input with a fixed-position pivot.

Optimized Solution: Randomized Pivot Selection

Replacing the fixed-position pivot with a uniformly random element (chosen fresh at every partition step) removes the correlation between the pivot choice and the data's existing order. No specific input arrangement — including a leaderboard that is already sorted or nearly sorted from the previous cycle — can reliably force an unbalanced split anymore, since the pivot is randomized independent of the data. This gives an expected $O(n \log n)$ running time regardless of input pattern, with only a vanishingly small probability of hitting the true worst case.

Design and Justification

Proposed design: implement randomized-pivot Quick Sort for the core re-sort operation, combined with an incremental-update strategy — since typically only one or a few scores change between display refreshes, a full re-sort every time is wasteful; instead, the changed entries can be removed and re-inserted into their correct position (an insertion-sort-like adjustment) when only a few scores change, falling back to a full randomized Quick Sort re-sort when

many scores change at once (e.g., end of a scoring round). As a further safety net, an Introsort-style approach — monitoring recursion depth and switching to Heap Sort if it exceeds a safe threshold — can be layered on top of randomized Quick Sort to provide a hard worst-case guarantee without sacrificing Quick Sort's typical-case speed.

Why randomized pivot selection is the right improvement: it directly targets the specific failure mode identified above (sorted/near-sorted leaderboard data defeating a fixed pivot), it requires only a small, cheap code change (one random index draw per partition), and it preserves Quick Sort's core appeal — low overhead, in-place operation, and fast typical-case performance — while removing the specific worst-case risk that matters most for a system that is re-sorted this frequently.

Case Study 3: Search Engine Query Processing (Binary Search)

Scenario: A search engine maintains a sorted index of keywords and must retrieve results quickly.

Analysis: How Binary Search Improves Search Efficiency

A search engine's keyword index can contain hundreds of millions of distinct terms, and every user query must be resolved to its matching index entry (or entries) in a fraction of a second, at massive query volume. Scanning such an index sequentially would be far too slow at this scale; because the index is maintained in sorted order, Binary Search can locate any keyword by repeatedly halving the search space, giving logarithmic-time lookups even as the index grows into the hundreds of millions of entries.

Key Requirements Identified

Sorted data: Binary Search's correctness depends entirely on the index being kept in sorted (typically lexicographic) order at all times — this is a structural requirement the search engine's indexing pipeline must guarantee, usually by building index segments in sorted order and merging them (itself often done with a Merge-Sort-style merge) rather than inserting into the middle of a live sorted array.

Fast retrieval at scale: With potentially billions of queries per day across a huge index, even small constant-factor inefficiencies compound enormously. Binary Search's $O(\log n)$ bound keeps the number of comparisons needed astonishingly small even for enormous indexes — around 27 comparisons for an index of 100 million keywords — making it practical to meet strict latency targets.

Applying Divide and Conquer

- **Divide:** examine the middle keyword of the current search range and compare it (lexicographically) to the query term.
- **Conquer:** if the middle keyword matches, the entry is found; if the query term is lexicographically smaller, recurse into the left half of the range; if larger, recurse into the right half.
- **Combine:** no combine step is needed — each recursive halving directly narrows down to the answer, discarding the half of the search space known not to contain the term.

Time Complexity and Performance Analysis

Case	Complexity	Reasoning
Best Case	$O(1)$	The query term happens to be exactly the middle element checked.
Average Case	$O(\log n)$	Each comparison eliminates half the remaining search space regardless of where the term lies.
Worst Case	$O(\log n)$	Even when the term is at either extreme or absent entirely, the search space still shrinks to a single candidate within $\lceil \log_2 n \rceil$ steps.

Space complexity is $O(1)$ for the standard iterative implementation, an important property when the search must run efficiently across enormous numbers of concurrent queries without per-query memory overhead scaling with index size.

Design and Justification

Proposed design: store each index segment as a sorted array (or a structure built on the same principle, such as a B+ tree, which generalizes binary search to reduce disk/cache reads by comparing against multiple keys per node instead of one). New documents are indexed into fresh, unsorted batches, which are then sorted and merged into the existing sorted segments periodically (a natural application of the same Merge Sort idea from Case Study 1) rather than being inserted directly into a live sorted array, avoiding the $O(n)$ shifting cost that a naive sorted-array insertion would incur. Query resolution then always operates against this maintained sorted structure using Binary Search (or its B+ tree generalization for disk-resident data).

Why Binary Search is optimal here: the index is large, relatively static between merge/update cycles, and queried extremely frequently — exactly the profile where Binary Search's $O(\log n)$ lookup, $O(1)$ space, and simple, predictable behavior outperform any linear alternative, and where the one-time cost of keeping the index sorted is easily justified by the enormous number of fast lookups it enables afterward.

Case Study 4: Image Processing System (Matrix Multiplication)

Scenario: An image processing system performs transformations using large matrices.

Analysis: How Divide-and-Conquer Matrix Multiplication Improves Performance

Many image processing operations — geometric transformations, convolution-based filters, color-space transformations, and machine-learning-based enhancement — are naturally expressed as matrix multiplications, often on very large matrices for high-resolution images or batches of images. The standard triple-loop multiplication algorithm runs in $O(n^3)$ time, which becomes a serious bottleneck as image resolution (and therefore matrix size) grows. Strassen's divide-and-conquer algorithm reduces this exponent, offering a genuine asymptotic improvement for large matrices.

Computational Challenges in Large Matrix Operations

- Cubic growth: standard multiplication's $O(n^3)$ cost means doubling matrix dimensions (e.g., processing a higher-resolution image) multiplies the computation time by 8, quickly becoming a bottleneck for high-resolution or batch processing pipelines.
- Memory bandwidth: large matrices demand significant memory bandwidth to move data between memory and processor, often becoming the practical bottleneck even before raw arithmetic throughput does.
- Real-time constraints: interactive image editing or live video processing requires transformations to complete within tight time budgets, making even modest constant-factor speedups valuable at scale.

Applying Divide and Conquer (Strassen's Method)

- Divide: each of the two input matrices is split into four equally sized quadrants (assuming, or padding to, a power-of-two dimension).
- Conquer: instead of the 8 sub-matrix multiplications a naive recursive approach would need, Strassen's method cleverly combines the quadrants using addition and subtraction to require only 7 recursive multiplications (M1 through M7), trading some of the multiplication cost for extra (cheaper) addition/subtraction operations.
- Combine: the four quadrants of the result matrix are reconstructed from specific sums and differences of M1–M7, and the four quadrants are assembled into the final product matrix.
- This recursion continues until the sub-matrices are small enough that a direct (standard) multiplication is cheaper than the recursive overhead — a practical base-case threshold used in real implementations.

Time Complexity Comparison

Method	Time Complexity	Notes
Standard Multiplication	$O(n^3)$	Simple, low overhead, but grows the fastest of the two as matrix size increases.
Strassen's Method	$O(n^{\log_2 7}) \approx O(n^{2.807})$	Asymptotically faster for large n , but carries extra overhead from recursive splitting and the additional addition/subtraction operations.

Because Strassen's recursive overhead outweighs its asymptotic advantage for small matrices, the crossover point where it becomes genuinely faster than standard multiplication occurs only once matrices are reasonably large — practical implementations exploit this by using Strassen's recursive splitting down to a threshold size, then switching to standard multiplication for the base case.

Design and Justification

Proposed design: implement a hybrid multiplier that recursively applies Strassen's decomposition for matrices above a tuned size threshold, falling back to standard (or a further block-optimized standard) multiplication below it — combining Strassen's asymptotic advantage on large transformations with standard multiplication's lower overhead on

small ones. For an image processing pipeline that repeatedly applies large-matrix transformations (e.g., batch processing many high-resolution images, or applying the same large transformation matrix across many frames of video), this hybrid can be further parallelized, since Strassen's seven sub-multiplications are independent of each other and can be computed concurrently on multiple cores or GPU threads.

Why this is efficient: the asymptotic saving compounds significantly at the large matrix sizes typical of high-resolution image data, and the hybrid threshold ensures the algorithm never performs worse than standard multiplication even on the smaller matrices that occur at the base of the recursion — giving the best of both approaches rather than committing fully to either one.

Case Study 5: Data Analytics Platform (Merge Sort vs Quick Sort)

Scenario: A data analytics platform must process large datasets efficiently for reporting.

Analysis: Using Merge Sort and Quick Sort in This Scenario

A data analytics platform typically needs to sort data for two rather different purposes: generating multi-column reports (where the relative order of tied records often carries meaning) and general-purpose in-memory sorting for computation speed (where raw throughput matters most and tie-breaking order is irrelevant). These two purposes pull toward different algorithms, making it worth analyzing both rather than committing to a single one.

Key Issues Identified

Memory usage: Merge Sort requires $O(n)$ auxiliary space for the temporary arrays used during merging, which can be significant for datasets that are already close to the platform's memory limits. Quick Sort, sorting in place, requires only $O(\log n)$ auxiliary space for its recursion stack — a meaningfully smaller footprint at large scale.

Speed: Quick Sort's in-place partitioning generally has a lower constant factor than Merge Sort's allocate-and-merge approach, making it faster in practice on typical in-memory random data, even though both share the same $O(n \log n)$ asymptotic complexity.

Stability: Analytics reports frequently sort by one column while needing to preserve a meaningful secondary order among ties (e.g., sorting transactions by amount while keeping same-amount transactions in their original chronological order). Merge Sort is stable and preserves this automatically; standard in-place Quick Sort is not stable and can silently scramble tied records' relative order.

Applying Divide and Conquer to Compare Both

Aspect	Merge Sort	Quick Sort
Where the 'work' happens	In the combine step (merging two sorted halves)	In the divide step (partitioning around a pivot before recursing)
Split strategy	Always splits by position (the midpoint) — value-independent	Splits by comparing to a pivot value — data-dependent
Guaranteed balance	Always perfectly balanced ($n/2$, $n/2$)	Depends entirely on pivot quality; can be highly unbalanced
Extra space needed	$O(n)$ for merging	$O(\log n)$ for recursion stack (in-place partitioning)

Performance Under Different Conditions

Condition	Better Choice	Reasoning
Data fits comfortably in memory, no stability needed	Quick Sort	Lower constant factor and smaller memory footprint typically make it faster in practice.
Data must be processed via external/disk-based sorting (too large for RAM)	Merge Sort	Its sequential merge pattern is naturally suited to combining sorted runs from disk.
Multi-column reports where tie order must be preserved	Merge Sort	Its inherent stability preserves secondary ordering automatically.
Data may already be sorted or nearly sorted (e.g., time-series logs)	Merge Sort (or randomized-pivot Quick Sort)	Guarantees $O(n \log n)$ regardless of input order; a naive Quick Sort risks worst-case $O(n^2)$ on such input.

Design and Justification

Proposed design: rather than picking one algorithm platform-wide, select based on the specific job's requirements. For ad-hoc, in-memory numeric sorts feeding a computation pipeline (e.g., sorting values before a statistical aggregation)

where tie order is irrelevant, use a randomized-pivot Quick Sort (or an Introsort variant with a Heap Sort fallback for worst-case safety) to take advantage of its speed and lower memory use. For generating user-facing, multi-column reports where secondary ordering must be preserved, or when sorting datasets too large to fit in memory, use Merge Sort (or an external k-way merge sort). A hybrid approach such as Timsort — which combines Merge Sort's stability and worst-case guarantees with efficient handling of already-ordered runs (extremely common in real-world time-series and log data) — is also a strong practical default for a general-purpose analytics platform that cannot predict in advance which kind of data each job will process.

Why this selective approach is justified: no single algorithm dominates on every axis — Quick Sort wins on raw speed and memory footprint, Merge Sort wins on stability and worst-case guarantees, and their relative advantage depends on data size, memory availability, and whether tie-order matters. Matching the algorithm to the specific job's requirements, rather than defaulting to one choice everywhere, ensures the platform gets Quick Sort's efficiency where it's safe to use and Merge Sort's reliability where correctness of the report depends on it.